

# Integrate ChatGPT API for AI-Assisted CV Building

## Overview

Integrating OpenAI's ChatGPT API into OkBuddy's CV builder is a critical enhancement to **accelerate and improve resume creation**. By leveraging a Large Language Model (LLM) at key steps of the user journey, OkBuddy can help users build a stronger CV faster and with less effort. The ChatGPT integration will power several features: analyzing job descriptions to tailor a CV, generating or improving resume content (work experience bullet points and summary), and providing intelligent suggestions throughout the CV editing process. The goal is to **embed AI assistance seamlessly** into the workflow so that users overcome writer's block and create high-quality, ATS-friendly resumes with ease.

## Use Cases & Integration Points

We will integrate the ChatGPT API at multiple touchpoints in the CV creation/editing flow:

- **Job Description (JD) Analysis – Tailoring CV to a specific job:** When a user provides a job description (either during the CV upload step or later in the Guided Editing page), the system will call the LLM to analyze the JD in context of the user's CV. This analysis extracts the *role's key requirements and keywords* and compares them against the user's current CV content. Based on this, the AI can suggest what skills or experiences the user should add or emphasize to better match the job. For example, if the JD repeatedly mentions "project management" and the CV lacks that phrase, the AI might recommend incorporating that keyword. The JD analysis results will be used to provide **tailored suggestions** in the Guided Editing interface (e.g. highlighting missing keywords or suggesting new bullet points relevant to the target job).
  - *Flow:* On the **CV & JD Upload page**, if a user uploads a JD along with their CV, the backend triggers an LLM call to perform this analysis while the user sees an "Analyzing your CV" loading screen. Similarly, if a user later enters a JD in the editor (or selects a job posting to target), the system will asynchronously run the analysis in the background.
  - *Output & UI:* After analysis, the Guided Editing page will present insights such as a list of important **keywords from the JD** not present in the CV, suggestions like "Consider adding experience with \_\_\_ (a skill/tool from the JD)" or an **ATS alignment score**. These suggestions will be displayed in a sidebar or highlighted

within relevant sections of the CV editor. They should be phrased as helpful tips, e.g., “Job description mentions **SQL** – you might want to add a project involving databases.” The UI will clearly indicate these as AI-driven recommendations (perhaps with an “AI Tip” label). If no JD was provided, the editor will still function, but with more generic improvement suggestions; we will *encourage users to add a JD* by noting “Thêm mô tả công việc sẽ giúp CV của bạn được tối ưu tốt hơn” (Adding a job description will improve tailoring of your CV) .

- **Resume Summary Improvement (“Cải thiện tóm tắt”) – *Polishing the Summary section*:** On the Guided Editing page, users can get AI help to improve their resume summary (the brief personal statement at the top of the CV). If the user has written a draft Summary (or generated one via AI), a one-click “**Improve Summary with AI**” button will be available. When clicked, the system sends the current summary text to ChatGPT along with contextual info like the user’s work experiences and the target job (if a JD is provided) . The LLM will return a refined version of the summary that enhances clarity, impact, and keyword alignment . It will ensure the summary highlights the user’s key strengths and matches the desired role/industry.
  - *Trigger:* User clicks the “Cải thiện tóm tắt” button (e.g., a magic wand icon next to the Summary field). This button is enabled only when there is some text in the Summary field (written by user or auto-generated).
  - *Output & UI:* The improved summary text is shown in place of the old one (or in a side-by-side comparison mode). We may highlight the changes or show the new text with a subtle background to indicate it was modified by OkBuddy AI. The user can accept it as-is or make manual edits. We will also display a note like “ Improved by OkBuddy AI” to build trust that the change is positive. If the summary was empty, the user would use the generate feature instead (covered below). If the user has not yet filled in any work experience or skills, the improvement might be limited – in such cases the AI might prompt the user for one or two key points to include (e.g., ask “What is your career goal or top strength?”) before generating a better summary .
- **Work Experience Quick Generation (“Tạo nhanh”) – *One-click drafting of a bullet point*:** In each Work Experience section, users will have an option to “**Quick Generate**” content based on minimal input, such as a job title or role. Initially, this feature simply took the job title and tried to produce generic bullet points, which led to irrelevant results if user context was missing . We are enhancing this with a *contextual prompt wizard*: when the user clicks “Tạo nhanh” for a given job entry, the system will guide them through a brief 2-3 step input process to gather context before calling the LLM. For example, the wizard might ask:

- “What was a key project or achievement in your role as **[Job Title]**?” (User enters a short answer, e.g. “Implemented a new inventory management system.”)
- “What was the impact or result of that project? (Any quantifiable outcome?)” (User might answer “Reduced stock outages by 30% and saved \$50K annually.”)
- (Optional) “Any specific tools/skills you used that you want to mention?” (User could skip or say e.g. “Used SAP ERP and Excel.”)

These inputs give the LLM a personalized context to generate a highly relevant bullet point . If the user skips the wizard or isn’t sure about answers, we will provide a **template-based fallback**: e.g. show a bullet template like “Led [team/task] to [outcome], resulting in [impact].” where the user can fill in the blanks . We can also leverage AI-powered autocomplete to help complete these blanks as the user types (suggesting phrases based on common responsibilities).

- *LLM Generation*: Once the user provides some context (or fills a template), the system formulates a prompt for ChatGPT that includes the user’s inputs and asks for a well-phrased bullet point. For example, the prompt (internally) might be: *“Generate one resume bullet point for a CV based on: Role = Sales Intern; Achievement = implemented a new inventory system; Result = reduced stock outages 30%, saved \$50k; Skills = SAP, Excel. The bullet should start with a strong action verb and quantify the result.”* The LLM’s output is expected to be a single bullet sentence, or possibly 1-2 bullets if we request multiple.
- *Output & UI*: The generated bullet is inserted into the Work Experience section for that job. It might appear highlighted as new content. We will append an “AI-generated” label or icon, and *keep the “Improve with AI” option available* in case the user wants to further refine it . The user remains in control – they can edit the bullet or delete it if it doesn’t fit. By involving the user via the mini-wizard, we ensure the content is grounded in their actual experience, avoiding the generic/unrelated output that a one-click blind generation might produce . This interactive flow should **boost user trust** in the AI suggestions, since the AI is now *“collaborating”* with the user rather than guessing blindly.
- *Edge Case - No Input*: If even with the wizard the user provides very little (or the section is completely blank), the system can still attempt a generic generation using just the job title, but we will caution that the result may be generic. It’s always preferable to have user input; thus the UI might show a hint like “💡 Tip: Provide a specific project or result for better suggestions.”
- **Work Experience Improvement (“Cải thiện với OkBuddy AI”) – Refining an existing bullet**: For any work experience bullet point that the user has written (either manually or via the generation feature), OkBuddy will offer a one-click **“Improve with AI”** action.

This uses the LLM to rewrite the selected bullet for improved clarity, impact, and inclusion of relevant keywords (especially if a JD was provided) . The original meaning and facts provided by the user are preserved, but the wording becomes more dynamic and results-driven. For instance, if a user wrote “Responsible for managing project X”, the AI could improve it to “Managed project X from start to finish, delivering results 2 weeks ahead of schedule and 15% under budget.”

- *Trigger & Scope:* Next to each bullet (or each experience entry) in the editor, an “Improve” icon (e.g. a magic wand ✨) will be shown. The user clicks it to enhance that specific piece of text. The system sends the bullet text to ChatGPT, along with context: the job title/industry (so the AI can incorporate domain-specific action verbs or jargon), possibly the overall CV or JD context for keywords (e.g., if the JD emphasizes certain skills, ensure the improved bullet includes them if relevant).
- *Output & Versioning:* The improved text is returned and replaces the original bullet in the UI. To reassure the user, we might keep the original text on hand (allowing an “undo” if the user preferred their version). Typically, however, the AI-improved bullet should be immediately beneficial: more concise, uses strong language, and adds quantifiable impact . This feature essentially acts as an **instant copy-edit and optimize** function, letting users polish their CV lines with minimal effort. Users can invoke it multiple times on the same bullet if needed (though we should discourage excessive cycles to save on cost; one improvement is usually enough, or we can advise the user to edit manually if not satisfied after one use).

## Prompt Engineering & Localization

To make the LLM integration maintainable and effective, all AI **prompt templates will be carefully engineered and externalized** for easy tweaking and localization. We will **not hard-code long prompts in the code**; instead, each prompt (or system message) is stored in a configuration file (e.g., a JSON or YAML) along with a version number and language tag. This approach aligns with how we handle UI text in multiple languages (storing in resource files) and allows prompt reuse and iterative improvement without code changes.

- **Structured Prompt Templates:** Define separate prompt templates for each scenario – e.g. JD\_ANALYSIS\_PROMPT\_V1, SUMMARY\_GEN\_PROMPT\_V1, SUMMARY\_IMPROVE\_PROMPT\_V1, EXP\_GEN\_PROMPT\_V1, EXP\_IMPROVE\_PROMPT\_V1. Each contains a template string with placeholders for dynamic data (user’s inputs, CV content, JD keywords, etc.). For example, the Summary-generation prompt might be: *“Write a 3-4 sentence professional summary for a resume using the following information about the user: list of inputs. The tone should be enthusiastic and tailored to target role, in {language}.”* The placeholders (like {language})

can be filled in at runtime or we use separate templates per language. Version numbers in keys (V1, V2) allow us to keep track of changes; if we find a better prompt formulation later, we can create SUMMARY\_GEN\_PROMPT\_V2 without completely overwriting V1, enabling A/B testing or rollback if needed. All prompt files will be loaded at startup so that **product managers or prompt engineers can update text or parameters easily** without diving into application code. Prompt version control also helps us measure improvements in output quality over time.

- **Localization & Language Handling:** Since OkBuddy serves both Vietnamese and English users (UI is in Vietnamese by default, but some users may have English content), the LLM prompts and outputs must adapt to the user's language. We will maintain prompt templates in at least Vietnamese and English. The system will choose the prompt language based on the user's interface language or the language of their CV content. For instance, for a Vietnamese user we use a Vietnamese prompt to instruct the LLM to answer in Vietnamese. Conversely, if the user's CV is in English, we use the English prompt variant. In practice, this can be handled either by separate localized prompt templates or by including a parameter like <language> in the prompt that explicitly tells ChatGPT which language to respond in. All AI-generated text shown to the user (suggestions, bullets, summaries) should appear in the appropriate language without manual translation. By localizing prompts, we ensure the **tone and style** of outputs feel natural in that language (especially important for Vietnamese where direct translations might seem awkward).
- **Consistency and Reuse:** The prompt config will also allow reuse of certain sub-prompts or instructions. For example, we might have a standard instruction for "the response format" or "make sure to include quantifiable achievements" that is used in multiple prompts. Rather than duplicating this text, the config could have a shared snippet (or the code assembles the final prompt from smaller pieces). This ensures consistency – e.g., every improvement prompt might append the instruction "Preserve the original meaning and facts, and use a confident professional tone." We maintain these guidelines uniformly across features.
- **Prompt Parameters & Tuning:** The integration will allow passing certain parameters to the ChatGPT API for each use case, which we will document in the config. For instance, the temperature setting might be lower (e.g., 0.3-0.5) for summary or bullet improvements to ensure the tone remains formal and deterministic, whereas for initial generation of content (blank page scenarios) we might allow slightly higher creativity (e.g., 0.7) but still not too high. Maximum tokens for responses will be tuned to expected output length: e.g., summary generation prompt can allow up to ~150 tokens output (since summaries are short), while JD analysis might need more tokens if it returns a list of suggestions. These parameters will also be adjustable in the config or code, and we will version them along with prompts if needed.

- **Versioning & A/B Testing:** By labeling prompts with versions, we enable safe updates. Suppose we find the JD analysis is not specific enough; we can craft a new prompt (V2) that perhaps asks the LLM to output results in bullet form with certain structure. We can test V2 for a subset of users (A/B testing) by configuring the system to use V2 prompt for X% of requests (this logic can be in the backend). By monitoring user engagement or satisfaction (or manually reviewing outputs), we decide if V2 is better before rolling out fully. This approach ensures continuous improvement of AI assistance quality without degrading user experience unexpectedly.

## Cost Efficiency Considerations

Cost efficiency is a paramount concern in using the ChatGPT API. We will adopt several strategies to manage API usage and expenses while delivering value:

- **Choosing the Right Model per Task:** OpenAI offers multiple model endpoints (e.g., gpt-3.5-turbo, gpt-4). We will use the **most cost-effective model that still meets quality needs** for each feature. For most generation and improvement tasks (bullet points, summary), the GPT-3.5 Turbo model should suffice with proper prompting – it is significantly cheaper and faster, making it suitable for interactive features where users may call it multiple times. We reserve GPT-4 usage for cases where higher reasoning or more nuanced output is absolutely required (for example, if JD analysis requires deeper understanding of complex job requirements or if a premium user specifically requests the highest quality). This tiered model approach ensures we control costs while not compromising output noticeably for the user. We can also make model choice configurable, so we can adjust or experiment over time (e.g., if OpenAI releases new models or pricing changes).
- **Prompt Optimizations:** We will keep prompts **concise and focused**, passing only the necessary information to produce good results. Unnecessary tokens in prompts (or including entire CV text when only a specific section is needed) will be avoided to reduce token counts. For example, if we are improving a single bullet, we do not need to send the user's entire CV or multiple bullets – just the target bullet, maybe the job title for context, and a brief instruction. Similarly, for JD analysis, rather than feeding the raw CV and JD directly (which could be very long), we might extract key sections or summarize the CV into a shorter form before sending to the LLM. We can use lightweight algorithms to identify skills in the CV and only send those plus the JD text, or ask the LLM to extract keywords from the JD and then just check which are missing from the CV via code instead of in the prompt. The spec leaves exact implementation to engineering, but emphasizes minimizing prompt size.
- **Batching and Caching:** Where possible, we will batch LLM requests or cache results to avoid duplicate calls. For instance, if the user uploads a CV and JD, we might call the LLM once to get a comprehensive analysis (e.g., missing keywords, suggested

improvements). Those results can then feed multiple UI components (keyword list, suggestions for each section) without separate API calls for each suggestion. If the user triggers an AI action repeatedly with the *same input* (say they click “improve” twice in a row without changing the text), we should recognize that and either avoid the second call or warn that the content is already optimized. Additionally, if two features can be combined in one prompt, we’ll consider that – e.g., when generating a summary, we might also ask the LLM for 2 variations in one call (instead of the user hitting regenerate which causes a second call). We will also explore caching common analysis results: for example, if two users input the exact same JD (common for popular job postings) along with similar profiles, the system could reuse the analysis. However, caching at this level must be done carefully due to personalization – likely more effective is caching *within a session* for that user (e.g., remember the JD analysis so we don’t call LLM again for the same JD unless the CV content changed).

- **User Usage Controls:** To protect against overuse by a single user (which can drive up cost), implement sensible limits or delays. For instance, for free users, we might limit the number of AI generations (they can improve maybe 5 bullets and generate 3 bullets for free, then prompt upgrade – depending on our monetization plan). Or if unlimited, ensure there’s a short cooldown between requests to prevent spamming the button rapidly. The spec doesn’t define exact pricing strategy, but notes that **free vs premium UX separation** is a principle – so perhaps basic AI features are free but heavy use or certain advanced AI (like GPT-4 or detailed cover letter generation) could be gated for premium subscribers. In any case, monitoring usage patterns will be important. We will log each API call with user ID and tokens used, so we can identify abuse or unexpectedly high usage and adjust accordingly (either by rate limiting or by refining prompts to reduce tokens).
- **Monitoring and Optimization:** We will set up dashboards or logs to track total tokens used per day and per feature. This will help identify which prompts are most costly. If one particular feature is consuming disproportionate resources, we can focus on optimizing it (e.g., maybe the JD analysis prompt is too verbose – we can try to shorten it or do more on our side). We’ll also watch latency: faster responses mean less likelihood of users retrying or getting impatient (which can lead to repeated calls). Using the stream API from OpenAI is another consideration – streaming partial responses for long outputs (if any) could improve perceived performance, though for our use cases, outputs are relatively short. Nonetheless, streaming could be useful if we later allow AI-generated cover letters or longer content.

## UX and Workflow Details

Seamless user experience is crucial when integrating AI features – we want the AI to feel like a natural assistant in the CV editing flow. Below are UX considerations and how the workflow will handle AI interactions:

- AI Activation & Loading Feedback:** Whenever the user triggers an AI action (JD analysis, generate, improve), the interface should clearly indicate that *something is happening* and set expectations. For example, after uploading a CV (and JD), we show a dedicated **analysis loading screen** with messaging like “ Analyzing your CV and job description...”. This screen includes either an animation or a progression indicator to keep the user engaged . Even if the backend AI processing finishes quickly, we will ensure the animation plays for a minimum duration (~3-4 seconds) so the user can absorb the feedback and not be jarred by a too-rapid transition . Conversely, if analysis takes longer than a few seconds, we will either loop a spinner or show progress stages (“Scanning your experience... Optimizing for the job...”) to reassure the user that work is ongoing . For in-editor actions like “Generate” or “Improve”, a smaller inline loading spinner or a placeholder text (e.g., “Generating suggestion...”) will appear near the button or in the text area. The buttons themselves can be disabled during processing to prevent duplicate clicks.
- AI-Generated Content Presentation:** Content created or modified by the AI should be inserted in a way that highlights it for the user’s awareness, but not in a jarring way. We will use subtle visual cues: for instance, newly generated bullet points might be highlighted with a light background color or accompanied by a ✨ icon. We might also display a tooltip or small text like “Generated by OkBuddy AI – please review and edit as needed.” This educates the user that they should still personalize the content. In the summary section, if the AI generates a paragraph, it could appear directly in the summary field with italic placeholder text until the user confirms it. We will make sure the user has to perform an action (like clicking “Accept” or starting to edit it, which implicitly accepts it) so they feel in control. The spec should note that any AI text is just a draft suggestion; the user can tweak or even discard it. Maintaining the user’s sense of ownership is important, as noted by user feedback (they don’t want the AI to impose content) . By keeping them in the loop (through the wizard input and the ability to edit), we ensure the AI is a co-pilot, not an auto-pilot.
- Multi-Step AI Flows:** Some AI features involve multiple steps (like the quick generation wizard). The UX for these should remain **lightweight and optional**. The wizard questions will be presented in a modal or side panel, one at a time, with a progress indicator like “Step 1 of 2”. Each question should be concise, with an example answer in grey text to guide the user. Provide skip options: at any point, the user can skip remaining steps if they don’t have the info, in which case we either use whatever input was given so far or fall back to templates. If the user cancels the wizard entirely, no content is generated and no API call is made (to save cost). We anticipate that this interactive flow will actually speed up content creation by helping users think of their achievements, and it addresses the earlier problem that one-shot generation was too generic . The spec should ensure the design of this flow is intuitive and doesn’t feel like a long form – **it should feel like a quick chat with the AI assistant**.



- **Guided Editing Suggestions:** Beyond the on-demand actions, the integration will allow the editor to proactively guide the user. For example, after the initial CV analysis, when the user enters the guided editing page, we can display a “**Guidance Panel**” or checklist. This might say: “OkBuddy AI Suggestions for You:” and list things like:

- *“Add 2-3 missing keywords from the job posting (e.g., Leadership, Project Management).”*
- *“Improve the summary to mention your 5 years of experience in Marketing.”*
- *“Consider adding a bullet about [Specific Skill] since the JD values this.”*

Each suggestion might have a one-click action next to it, like a button “Apply” or “Auto-fix” which, if clicked, triggers the relevant AI feature. For example, if the suggestion is about missing keyword X, clicking it could auto-invoke the “Improve with AI” on the most relevant bullet, instructing the AI to incorporate X. Or it could start a new bullet generation focused on X. This gives users a roadmap to follow, especially helpful for those who aren’t sure where to start. It also increases feature discoverability (users see what the AI can do for them).

- **User Control and Editing:** In all cases, the user retains full control over the final content. After AI generates or modifies text, the user can directly type in that field to adjust it. All editor fields (summary textarea, experience bullet inputs) will remain editable after AI insertion. We will not lock any AI-generated content. If the user edits the AI text, we consider that final – we won’t override their changes unless they explicitly click “Improve” again. If they do request another improvement after editing, the new prompt will then include their edited version as input (so the AI works off their refined text). Essentially, it’s an iterative loop where the user and AI can take turns refining until the user is satisfied.
- **Error Feedback (UI):** If an AI action fails (due to network or API error), the user should be informed in a friendly way. For instance, if “Generate bullet” fails, we might show a small error message below the button: “Oops, the AI is taking a break. Please try again in a moment.” Maybe include a hint to check internet or that the system will retry shortly. Importantly, if an error occurs during the CV upload analysis (rare, but possible if our server cannot get a response), we **must not block the user from proceeding**. The loading screen in that case would timeout gracefully into the editor even without tailored suggestions. We can show a brief alert in the editor like “We couldn’t run AI analysis at this time, but you can still edit your CV. Try reloading the job description later.” This ensures the user’s flow isn’t dead-ended by an AI issue.
- **Performance Considerations:** While using the ChatGPT API, we need to keep the app feeling responsive. Most calls should complete in a couple of seconds. If some requests (like a very large JD analysis) might take longer, we should consider doing those in

parallel or background. The initial CV analysis can be done while the user is viewing the animated loading screen (with a minimum delay as noted). For in-editor calls, if they take too long (say >5 seconds), we should allow the user to cancel. Perhaps the “Generating...” message can have a small cancel “✖” icon to abort the request if the user doesn’t want to wait (this will simply stop showing the result if it arrives later). However, cancellations should be rarely needed if we optimize well.

- **Testing & Quality Assurance:** We will test the AI features extensively with different input cases (varied job titles, industries, languages). Particularly, we’ll verify that for Vietnamese inputs, the outputs are correctly in Vietnamese and use proper professional phrasing. We’ll also test edge cases like extremely short inputs, or very long JD texts. The prompts may need refinement based on testing – e.g., we might find the AI sometimes outputs too long a summary; we then adjust the prompt to emphasize a limit (as seen in our summary example prompt asking for 3-4 sentences ). This spec expects iterative prompt tuning during QA. We should also involve some end users or team members in beta testing to gather feedback on the usefulness of suggestions and tweak accordingly.

## Error Handling & Edge Cases

While we aim to cover the typical flows, it’s important to handle edge cases and failures gracefully in both the AI integration and the user experience:

- **API Failure or Timeout:** If the ChatGPT API call fails (e.g., network issue, API returns an error, or times out), the system will catch this and **prevent any crash or freeze**. The user will either see the previous content unchanged or a partial result if we were streaming (though likely we use complete responses for these short texts). We will display a non-intrusive error message specific to the action. For example, if “Improve with AI” fails, show a tooltip or inline note: “Couldn’t get an improved suggestion at this time. Please try again.” The message should be localized (configurable text). In the background, we should log the error (with maybe the prompt ID and error code) for developers to analyze later. If appropriate, we could also implement an automatic retry mechanism for certain failures (e.g., rate limit hit – wait a second and retry once).
- **LLM Output Quality Issues:** Sometimes the AI might produce output that is irrelevant, incorrect, or exceeds the desired length. Our approach to mitigate irrelevance is the context-driven prompts (wizard input) , but if a bad suggestion does slip through, the user can simply delete or ignore it. It’s important that the AI *never auto-saves content without user review*. All AI outputs are essentially suggestions until the user accepts them. If an output is factually incorrect (e.g., it introduces a wrong detail), the user can edit it; we might also add a disclaimer in UI help text that “AI suggestions may need your review for accuracy.” For length issues, especially in summary, our prompt will explicitly limit output length, and we can also implement a check: if the returned text is, say, more

than 100 words for a summary, we automatically truncate it or ask the AI again with a stricter prompt. The improvement function should also ensure not to introduce new information that the user didn't provide (to avoid misleading statements on a CV). This is more of a prompt design concern – e.g., include “do not make up achievements, only rephrase given information” in the prompt.

- **User Inputs and Edge Cases:**

- If the user's CV is very sparse (e.g., only one job with one line) and they request a full summary or multiple bullet generations, the AI might struggle due to lack of info. In such cases, we have the **interactive fallback** questions (for summary: ask for career goal or strength ; for bullet: ask for an achievement) to solicit more input. If the user still provides nothing, the AI will generate very generic content. That content might not be great – but it's arguably better than nothing as a starting point. We will document in the UX that the more input the user gives, the better the output. If a user enters obviously irrelevant text or jokes into the wizard, the AI could produce irrelevant output – but that's user-driven, not much we can do except perhaps detect if inputs are too off-base and warn “Your answer seems unrelated – the suggestion may not be accurate.”
- If the JD provided is extremely long (some job posts are multi-page), sending it whole to the API could be expensive or hit token limits. If needed, we will **summarize or extract** the JD on our side first. We could use an algorithm to pick out the responsibilities and qualifications sections, or even call a cheaper model to summarize the JD into key points, then feed that to the main prompt. This two-step approach could save tokens. It adds complexity, but it's worth noting for implementation if we encounter limits. (The spec leaves the exact method open, but highlights that we should not naive-pass a huge text in one go if it breaks limits).
- If a user *does not upload a JD at all*: The JD analysis step is simply skipped. The “Analysis” loading screen may be shorter (just analyzing CV for general improvements). The AI can still do a generic CV analysis (like checking for common weaknesses). In fact, we could still use the LLM to identify general improvements even without a JD (e.g., “Your CV might benefit from more action verbs” or “Consider adding technical skills section”). However, those are secondary; the main value is with a JD. The UI will downplay missing JD – we don't want to “guilt” the user, but we might gently remind them that providing a JD yields better results . If no JD, any suggestions labeled as “tailored to JD” will not appear, obviously.
- If the user uploads a non-textual JD (like an image or PDF of a job post): Ideally, we should perform OCR or ask them to paste text. The spec can note that JD input field expects text. If it's a PDF file, we might parse text out of it in backend if

feasible, then feed to AI. This is an edge case depending on how the upload is implemented (the current design likely has a text box or file upload for JD; since the spec doesn't clarify, we should assume text input or PDF upload is possible). We will handle parsing as needed so the AI still gets text.

- **Privacy and Data Handling:** Sending user data (CV content, JD content, personal details) to an external API (OpenAI) has privacy implications. We must ensure this is covered in our Privacy Policy and that users consent (likely in terms of service) to their data being processed by an AI service for the purpose of getting suggestions. We won't use the data beyond the immediate analysis. Also, we should **mask any personal identifiable info** not relevant to the CV content when sending to the LLM. For example, the user's contact info (phone, email) in the CV isn't needed for analysis – we can exclude or anonymize that before sending the CV text. Similarly, if a user's CV has company names or such and we're concerned about confidentiality, we might generalize them (though likely not necessary unless user is worried). OpenAI does offer data usage policies; we should use their API in a way that **does not log or train on our data** if that option exists (OpenAI allows opting out of data being used for training). Engineering can handle that configuration.
- **Concurrent Usage:** If a user triggers multiple AI actions in rapid succession (e.g., clicks "Improve" on two different bullets quickly), our system should handle it – possibly queue the requests or process in parallel if threads allow. The main risk is overloading the API or hitting rate limits. We will implement modest **rate limiting** on our end: for instance, not allowing more than, say, 3 concurrent LLM requests per user, and not more than e.g. 10 requests per minute sustained. If a user somehow manages to click very fast (or two users on the same account), additional clicks can be ignored or told "Please wait for current suggestions to finish." This prevents both cost runaway and hitting the OpenAI rate limits (which could then affect other users). If we do hit a global rate limit, we treat it similar to an error – let the user know to try again.
- **Logging & Analytics:** As part of error handling and continuous improvement, we will log significant events: when AI generation is used, whether the output was accepted or edited (if we can detect that), errors encountered, etc. This data can help refine prompts and also measure feature success (e.g., "30% of users are using AI improve on at least one bullet"). Logging should exclude sensitive content (or at least be secured) – maybe log the prompt ID and length, not the actual text for privacy. If we notice many errors or drop-offs at a particular step, that's a sign to revisit the spec or implementation (for example, if many users cancel out of the "Tạo nhanh" wizard, maybe it needs to be simplified or faster).

**Edge Case Summary:** In the worst-case scenario that the AI features are all failing (e.g., OpenAI API downtime), the system will still allow manual resume editing. We have designed the process so that AI assistance is beneficial but **not strictly required** to use OkBuddy. Users can

always proceed with writing their CV manually as a fallback. This ensures reliability: OkBuddy's core functionality is not crippled by AI outages. We simply lose the enhancement until it's back online, and we will communicate issues proactively if needed (e.g., a banner "AI features are temporarily unavailable, we're on it!" if there's extended downtime).